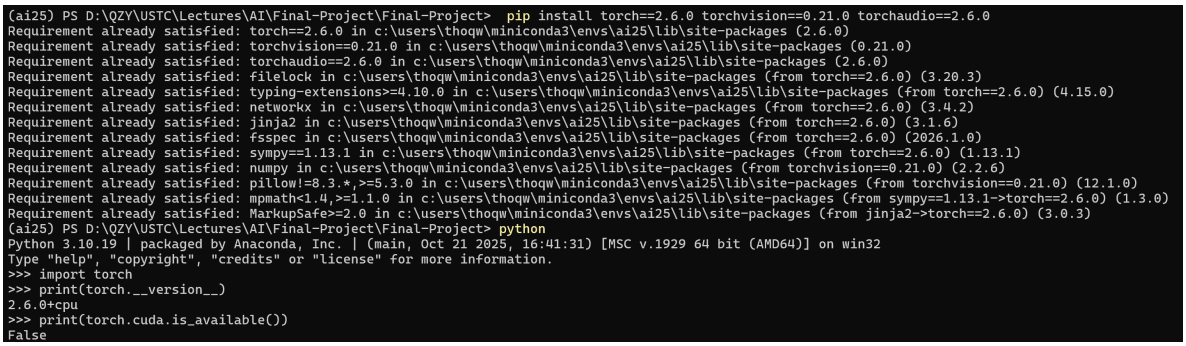
AI final project report

Zeyuan Qiu PB22000233 Runkun Liu PB24000201 Denghong Luan PB24000360

February 27, 2026

1 Torch Configuration



```
(ai25) PS D:\QZY\USTC\Lectures\AI\Final-Project\Final-Project>  pip install torch==2.6.0 torchvision==0.21.0 torchaudio==2.6.0
Requirement already satisfied: torch==2.6.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (2.6.0)
Requirement already satisfied: torchvision==0.21.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (0.21.0)
Requirement already satisfied: torchaudio==2.6.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (2.6.0)
Requirement already satisfied: filelock in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torch==2.6.0) (3.20.3)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torch==2.6.0) (4.15.0)
Requirement already satisfied: networkx in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torch==2.6.0) (3.4.2)
Requirement already satisfied: jinja2 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torch==2.6.0) (3.1.6)
Requirement already satisfied: fsspec in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torch==2.6.0) (2026.1.0)
Requirement already satisfied: sympy==1.13.1 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torch==2.6.0) (1.13.1)
Requirement already satisfied: numpy in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torchvision==0.21.0) (2.2.6)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from torchvision==0.21.0) (12.1.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from sympy==1.13.1->torch==2.6.0) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\thoqw\miniconda3\envs\ai25\lib\site-packages (from jinja2->torch==2.6.0) (3.0.3)
(ai25) PS D:\QZY\USTC\Lectures\AI\Final-Project\Final-Project> python
Python 3.10.19 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 16:41:31) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> print(torch.__version__)
2.6.0+cpu
>>> print(torch.cuda.is_available())
False
```

Figure 1: Torch Environment Configuration

We successfully installed PyTorch via pip, imported torch in Python, printed the version number and CUDA availability to confirm the completion of the environment configuration (see Figure 1 for Torch environment configuration).

2 warmup

In the warmup phase, we implemented the Monte Carlo (MC) learning, Q-learning and SARSA algorithms. The analysis of the experimental results is presented as follows.

2.1 MC Learning

Core Idea: The core principle of the Monte Carlo method is to replace the expected value with the empirical average. First-Visit MC specifically uses the cumulative return G_t generated from the first visit to a state s (or state-action pair (s, a) in an episode to update the value estimate. This ensures the independent and identically distributed (i.i.d.) property of samples, thus providing an unbiased estimate.

Update Target: First-Visit MC mainly updates the action-value function $Q(s, a)$. For each state-action pair (s, a),two variables are maintained:

- $Returns(s, a)$: A list (or cumulative sum) storing all cumulative returns from the first visits to the pair.

- $N(s,a)$: The count of first visits to the pair.

The update formula is:

$$Q(s,a) \leftarrow Q(s,a) + \frac{1}{N(s,a)}(G_t - Q(s,a)) \tag{1}$$

This is equivalent to $Q(s,a) = \text{average}(Returns(s,a))$。

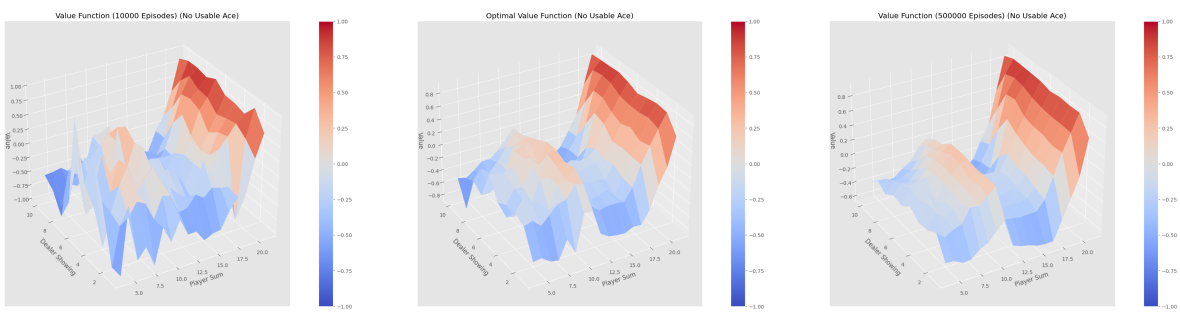
Policy Improvement:The ϵ-greedy policy is adopted for policy improvement:

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(s)|} & \text{if } a = \arg\max_{a'} Q(s,a') \\ \frac{\epsilon}{|A(s)|} & \text{otherwise} \end{cases} \tag{2}$$

where ϵ ensures exploration, enabling all actions to be selected with a non-zero probability and satisfying the sustained exploration assumption required by the Monte Carlo method.

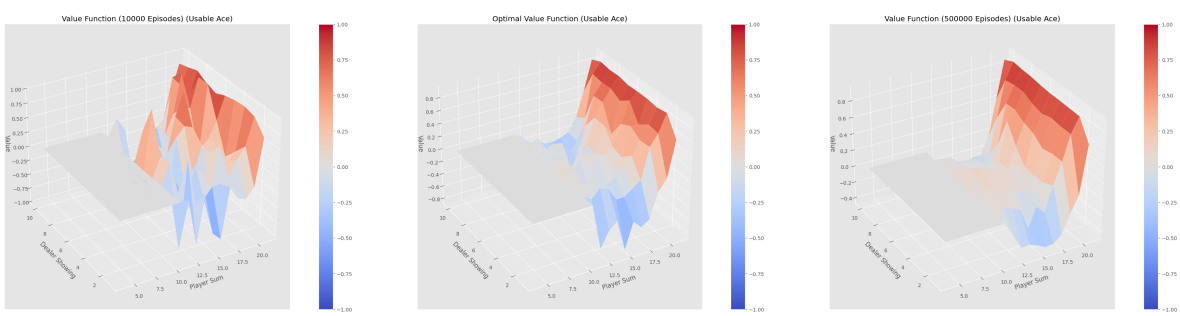
2.1.1 Comparison of Different Training Episodes

We conducted experiments with 10,000, 100,000 and 500,000 training episodes, and the corresponding training plots are obtained as follows:



(a) 10000 episodes (b) 100000 episodes (c) 500000 episodes

Figure 2: Comparison plots for different training episodes(no usable ace)



(a) 10000 episodes (b) 100000 episodes (c) 500000 episodes

Figure 3: Comparison plots for different training episodes (usable ace)

The plot for 10,000 episodes shows numerous spikes and a rough surface, indicating high noise and potential underfitting in this scenario. As the number of training episodes increases, the surface gradually becomes smooth and flat.

Since the MC method estimates the Q-value via sampling, insufficient samples will lead to a large variance in the estimates and difficulty in converging to the true value, resulting in rough and irregular curves. With the increase in the number of samples, the estimated state value gradually converges to the true expected return according to the Law of Large Numbers, making the curve smooth and close to the true value.

A usable ace (which can be counted as 1 or 11) provides the player with a second chance. Since hard totals of 12 to 16 lead to a steep decline in the plot for this range, the value of a usable ace is slightly higher than that of no usable ace.

2.1.2 Discussion on First-Visit MC

Convergence Characteristics

1. Unbiasedness: The value estimate for each state is based solely on independent episode samples (returns from first visits), and its expected value is strictly equal to the true value $v_\pi(s)$ with no systematic bias.

2. High Variance: The estimate relies on the cumulative return of complete long episodes and is significantly affected by the cumulative randomness of the environment and policy. The single-sampling fluctuation is extremely large, leading to obvious oscillations in the convergence process.

3. Convergence Speed: It follows the Law of Large Numbers, with the standard deviation decreasing at a rate of $\frac{1}{\sqrt{N}}$ (the Central Limit Theorem guarantees a standard deviation of $\frac{\sigma}{\sqrt{N}}$) as the number of samples N increases. Although unbiased, the large variance usually requires a massive number of samples for stable convergence.

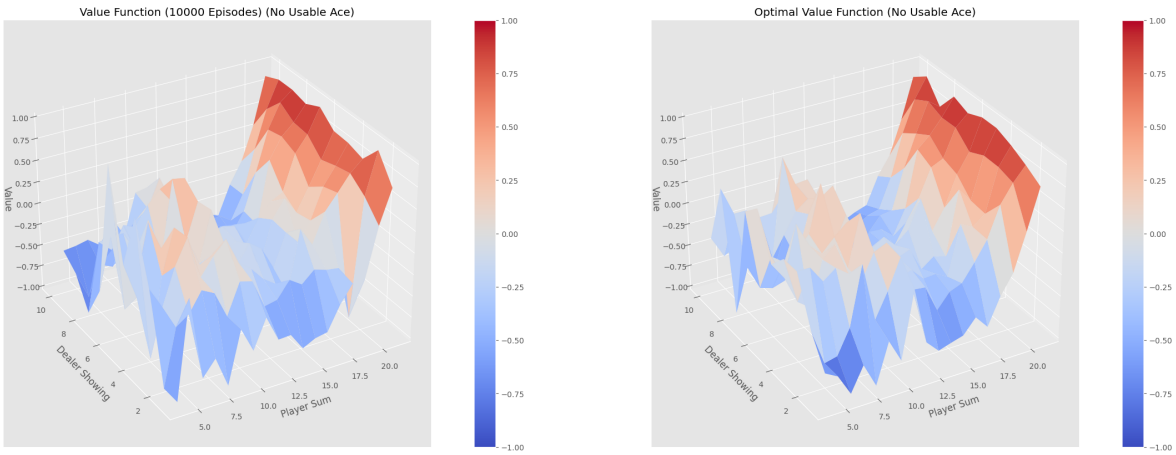
Sources of Variance

1. Randomness of Returns: The update target of MC is G_t, a multi-step cumulative quantity. In addition, the $\epsilon - greedy$ policy is used for each action update, leading to randomness in each step and the accumulation of randomness across multiple steps. Thus, even starting from the same state, the multi-step cumulative randomness results in significant differences in subsequent states.

2. Sparse Visits to Specific States: In environments with a large state space such as Blackjack, certain states (e.g., specific card combinations) are rarely encountered. For these "rare" states, the small sample size $N(s)$ causes extreme fluctuations (variance) in the average value.

3. Sample Size Limitation: Compared with Every-Visit MC, First-Visit MC only obtains one estimate per episode, and some episodes may be long, which limits the acquisition of sufficient samples and may lead to a larger variance.

2.1.3 Comparative Analysis with Every-Visit MC



(a) First-Visit MC (10,000 episodes)

(b) Every-Visit MC (10,000 episodes)

Figure 4: Comparison (no usable ace)

Theoretical Analysis

- **First-Visit MC**: It completely ignores the return information from subsequent revisits to the same state in an episode, making it insensitive to loops in state transitions. However, the estimates show large fluctuations with a small sample size, resulting in worse stability than Every-Visit MC.
- **Every-Visit MC**: It utilizes more data to smooth the estimates, thus generally exhibiting better stability (Variance Reduction).

Practical Performance in Blackjack

Blackjack has a unique property: the number of cards held by the player only increases, meaning the state evolves unidirectionally (a usable ace can be downgraded but cannot return to the original total). As a result, the same (PlayerSum,DealerCard,UsableAce) state is almost never revisited in a single episode. This implies that First-Visit MC and Every-Visit MC behave nearly identically in Blackjack, which is consistent with the results in the comparative plots—the actual difference between the two methods is negligible.

2.2 Temporal-Difference Learning

2.2.1 Update Differences

SARSA: When updating $Q(s, a)$, SARSA uses the actual next action a' executed in the environment. This means it learns the policy currently being followed (including exploration steps), i.e., SARSA is an on-policy algorithm.

Update formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R + \gamma Q(s', a') - Q(s, a)] \tag{3}$$

Characteristics:

- Considers the exploration risk in the ϵ-greedy policy
- In environments with dangerous regions (e.g., the cliff-walking environment), SARSA tends to choose safer paths to avoid falling into traps due to random exploration.

Q-learning: Q-learning When updating $Q(s,a)$ Q-learning assumes the optimal next action $\max_{a'} Q(s',a')$, is taken, regardless of the actual action executed. This means it learns the optimal policy while the behavioral policy (e.g.ϵ-greedy) can be arbitrary. Unlike SARSA, Q-learning is an off-policy algorithm.

Update formula:

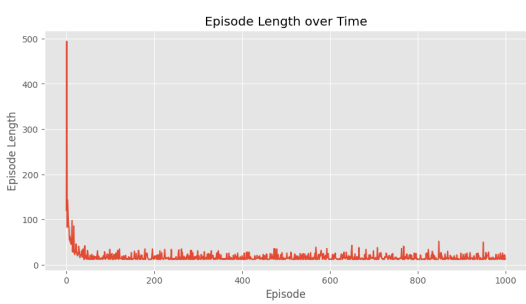
$$Q(s,a) \leftarrow Q(s,a) + \alpha[R + \gamma \max_{\mathbf{a}'} Q(s',a') - Q(s,a)] \tag{4}$$

特点:

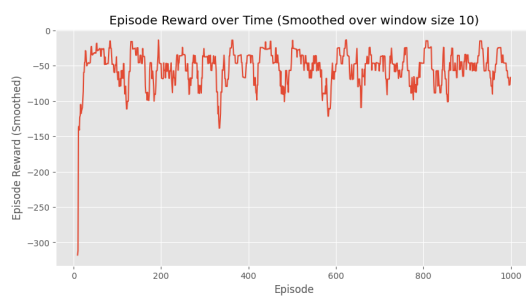
- Directly approximates the optimal value function Q^*.
- More aggressive: in the cliff-walking environment, it hugs the cliff edge (the optimal path). Although it may frequently fall off the cliff during training due to exploration, the final learned policy is the shortest path.

2.2.2 结果分析

We set the hyperparameters as follows: discount factor = 1.0, alpha = 0.5, epsilon = 0.1 and the training episodes = 1000. The curves of episode length and reward for Q-learning and SARSA are shown in the following figures:

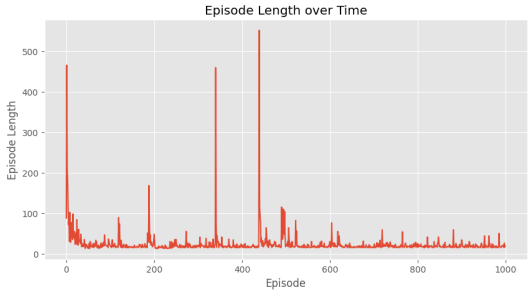


(a) Q-learning episode length

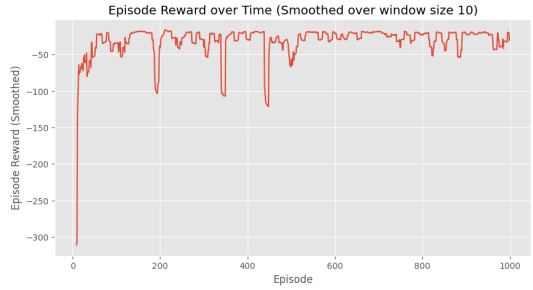


(b) qlearning episode reward

Figure 5: qlearning



(a) sarsa episode length



(b) sarsa episode reward

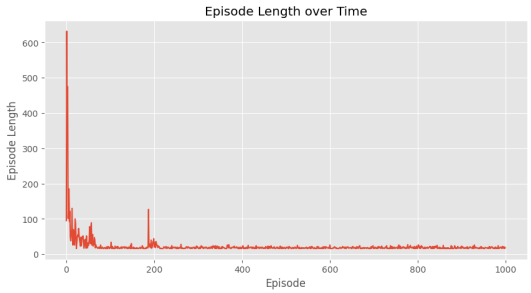
Figure 6: sarsa

analysis:

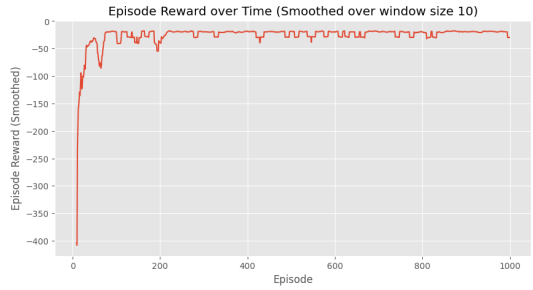
length: Both algorithms converge well overall. Although SARSA shows unstable episode length at around 400 episodes, the episode length of both algorithms converges to a low level after about 600 episodes, indicating their excellent convergence performance in terms of episode length. However, Q-learning aims to update the off-policy Q-value and thus adopts a more aggressive strategy. In the case of a "safe and long path", SARSA may not pursue the target as directly as Q-learning and may "linger" in the safe area due to randomness. This explains why SARSA is more likely to have extremely high episode lengths and a larger variance than Q-learning.

reward:For a large number of episodes, the reward of both SARSA and Q-learning hovers around -50. SARSA converges more stably and remains above -50 after 600 episodes, while Q-learning shows larger fluctuations. This is because Q-learning adopts a more aggressive strategy, leading to a higher probability of incurring greater losses or obtaining greater gains during exploration, thus resulting in a larger variance.

We also implemented Double Q-learning, and its performance is shown in the following figure:



(a) double qlearning episode length



(b) double qlearning episode reward

Figure 7: double q learning

Double Q-learning achieves the best performance among all algorithms: it converges the fastest and to a higher reward value, and the curve remains extremely stable after convergence —a significant improvement over Q-learning.

The superior performance of Double Q-learning can be explained by the overestimation bias of Q-learning: Q-value estimates are inherently biased (either overestimated or underestimated). Q-learning always selects the action with the maximum$Q[next_state][a]$during updates, so over-

estimated Q-values are more likely to be chosen, leading to a persistent overestimation of the true Q-value and inaccurate results. If a non-optimal action is randomly selected during Q-learning, the previously overestimated Q-value will drop sharply, causing unstable rewards.

Double Q-learning addresses this overestimation bias by maintaining two independent Q-value estimators (Q_1 and Q_2): if Q_1 is used to select the optimal action, Q_2 is used to evaluate its value; if Q_2 is used to select the optimal action, Q_1 is used to evaluate its value. The selection between the two cases is random during implementation. Due to the independence of Q_1 and Q_2, the probability of both overestimating the same action is low, leading to more accurate Q-value estimates and reduced curve fluctuations.

3 Main Experiment

3.1 Experimental Principles

3.1.1 Two-Player Zero-Sum Markov Game

A **Two-Player Zero-Sum Markov Game** refers to a competitive decision-making process where two players interact in a time-varying environment. Each action affects the subsequent state, and the gain of one player equals the loss of the other.

- **Two-player**: Only two players participate in the game.
- **Zero-sum**: The sum of the rewards for both players is zero:

$$r_1(s, a_1, a_2) = -\, r_2(s, a_1, a_2).$$

- **Markov property**: The next state only depends on the current state and the actions of both players:

$$s_{t+1} \sim P(\cdot \mid s_t, a_{1,t}, a_{2,t}).$$

3.1.2 Naive Self-Play

Naive Self-Play Naive Self-Play means an agent generates training data and updates its policy by continuously playing against the "current version of itself" (or the latest policy), thus gradually improving its performance in a competitive environment.

- Let the policy be $\pi_\theta(a \mid s)$, The training process of naive self-play repeatedly executes the following steps:

$$\text{rollout from } (\pi_\theta \text{ vs. } \pi_\theta) \;\Rightarrow\; \mathcal{D} \;\Rightarrow\; \theta \leftarrow \theta + \Delta\theta.$$

- advantages: Simple implementation; no external expert data required.
- Common Issues: Potential policy cycles (mutual restraint between policies), unstable training, and forgetting of old opponents.

3.1.3 Actor-Critic Method

Actor-Critic is a class of reinforcement learning methods that uses two separate models to learn the policy: the Actor for action selection and the Critic for action evaluation.

- **Actor**: $\pi_\theta(a \mid s)$, responsible for selecting actions to maximize the long-term cumulative return.
- **Critic**: $V_\phi(s)$ or $Q_\phi(s,a)$responsible for estimating the return. For the state-value function:

$$V^\pi(s) = \mathbb{E}_\pi\Big[\sum_{t=0}^{\infty} \gamma^t r_t \,\Big|\, s_0 = s\Big].$$

The Temporal-Difference (TD) error is usually used to approximate the advantage function:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t),$$

The Actor is updated via policy gradient and uses the advantage function to reduce variance. The update direction (maximizing the expected return) is:

$$\nabla_\theta J(\theta) = \mathbb{E}\big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A_t\big].$$

The Critic is updated by minimizing the squared TD error:

$$\mathcal{L}(\phi) = \mathbb{E}\big[\delta_t^2\big].$$

3.2 Model Design

3.2.1 Actor:(Policy Network)

Function and Input/Output

The Actor aims to generate the policy $\pi(a \mid s)$, enabling the agent to select an action a in state s to maximize the long-term cumulative return. In this implementation:

- Board state (shape: (N,N) or $(B,1,N,N)$,where B is the batch size and N is the board size)
- Policy probability distribution (shape:(B,N^2))

Key Code for Network Architecture

The Actor uses a convolutional neural network to extract spatial features of the board, followed by a fully connected layer to output action logits of dimension N^2.

```
N = board_size
# feature extractor
self.conv_blocks = nn.Sequential(
    nn.Conv2d(in_channels=1, out_channels=32,
              kernel_size=3, padding=1),
    nn.ReLU(),
    nn.Conv2d(in_channels=32, out_channels=64,
```

```
                kernel_size=3, padding=1),
    nn.ReLU(),
)

# policy head
self.linear_blocks = nn.Sequential(
    nn.Flatten(),  # shape: 64*N*N
    nn.Linear(64*N*N, 256),
    nn.ReLU(),
    nn.Linear(256, N*N),  # shape: N*N
)
```

Listing 1: Actor Network Architecture: Convolutional Feature Extraction + Fully Connected Policy Head

Forward Propagation

A critical design point of the Actor is to ensure a zero probability for illegal actions and re-normalize the probabilities of legal actions.

```
if len(x.shape) == 2:  # (N,N)
    board = torch.tensor(x).to(device).to(
        torch.float32).unsqueeze(0).unsqueeze(0)
else:  # (B,1,N,N)
    board = torch.tensor(x).to(device).to(torch.float32)

B = board.shape[0]

features = self.conv_blocks(board)
logits = self.linear_blocks(features)  # (B, N*N)


legal_mask = (board == 0).view(B, -1)  # (B,N*N)
masked_logits = logits.masked_fill(~legal_mask, float('-inf'))

output = torch.softmax(masked_logits, dim=1)

no_legal = legal_mask.sum(dim=1) == 0
if no_legal.any():
    output[no_legal] = torch.zeros_like(output[no_legal])

return output
```

Listing 2: Actor Forward Propagation: Input Processing + Legal Action Masking + Normalization

3.2.2 Critic: Q-value Network

Function and Input/Output

The Critic aims to estimate the action-value function $Q(s, a)$, which is used to fit the Bellman equation and guide the Actor's update.

- Input: board shape s (shape: (N, N) or $(B, 1, N, N)$) and action a(shape: $(B, 2)$)
- Output: Q-value for the corresponding action (shape: $(B,)$)

Key Code for Network Architecture

The feature extraction part of the Critic is similar to the Actor, but the output represents the Q-value for each action.

```
N = board_size
# feature extractor
self.conv_blocks = nn.Sequential(
    nn.Conv2d(in_channels=1, out_channels=32,
              kernel_size=3, padding=1),
    nn.ReLU(),
    nn.Conv2d(in_channels=32, out_channels=64,
              kernel_size=3, padding=1),
    nn.ReLU(),
)

# value head
self.linear_blocks = nn.Sequential(
    nn.Flatten(),
    nn.Linear(64*N*N, 256),
    nn.ReLU(),
    nn.Linear(256, N*N),
)
```

Listing 3: Critic Network Architecture: Convolutional Feature Extraction + Full-Action Q-value Output

Forward Propagation:

Index mapping + gather to extract the Q-value for the specified action

```
indices = torch.tensor(
    [_position_to_index(self.board_size, r, c) for r, c in action]).to(device)

if len(x.shape) == 2:
    board = torch.tensor(x).to(device).to(
        torch.float32).unsqueeze(0).unsqueeze(0)
else:
    board = torch.tensor(x).to(device).to(torch.float32)

features = self.conv_blocks(board)
q_values = self.linear_blocks(features)  # (B, N*N)

output = q_values.gather(1, indices.unsqueeze(1)).squeeze(1)  # (B,)
return output
```

Listing 4: Critic Forward Propagation: Extract Q(s,a)

3.3 Constrained Policy Analysis

In the Gomoku task, the Actor outputs a policy $\pi(a \mid s)$,$a \in [N^2]$,with the constraint that pieces cannot be placed on occupied cells. In addition, as a probability distribution, π needs to satisfy the normalization condition:

$$\sum_a \pi(a \mid s) = 1$$

.

Since the network generates logits $\in \mathbb{R}^{\mathbb{N}^{\not\models}}$, the softmax function is needed to map them into a probability distribution. If logits$= -\infty$, the corresponding probability is 0, so the probability of illegal actions can be set to 0 by naturally setting the logits of all occupied positions on the board to negative infinity:

$$\log \pi(a|s) = \begin{cases} \text{logit}_a & a \in \mathcal{A}_{legal}(s) \\ -\infty & a \notin \mathcal{A}_{legal}(s) \end{cases}$$

where $\mathcal{A}_{legal}(s)$ is a mask representing the positions where pieces can be placed.

The corresponding pseudocode is:

```
legal_mask = (board == 0).view(B, -1)
masked_logits = logits.masked_fill(~legal_mask, -ϖ)
```

The properties of the softmax function automatically satisfy the probability normalization condition. However, due to the possible accumulation of numerical calculation errors, the probability distribution is re-divided by the sum of the probabilities of all positions in the implementation.

3.4 Curve Analysis

Here we show the training curves of the optimized model.

3.4.1 Trend of Actor Loss

In the first 500 episodes, the Actor Loss drops rapidly from nearly 0 to about -0.5, indicating that the policy starts to favor actions with high advantage values. In 500-2000 episodes, the mean value stabilizes at around -0.5, but the variance (fluctuation) gradually increases, which may be affected by the noise of the Critic and the exploration policy. In 2000-3000 episodes, the mean value decreases further but with larger fluctuations, leading to reduced stability of policy updates.

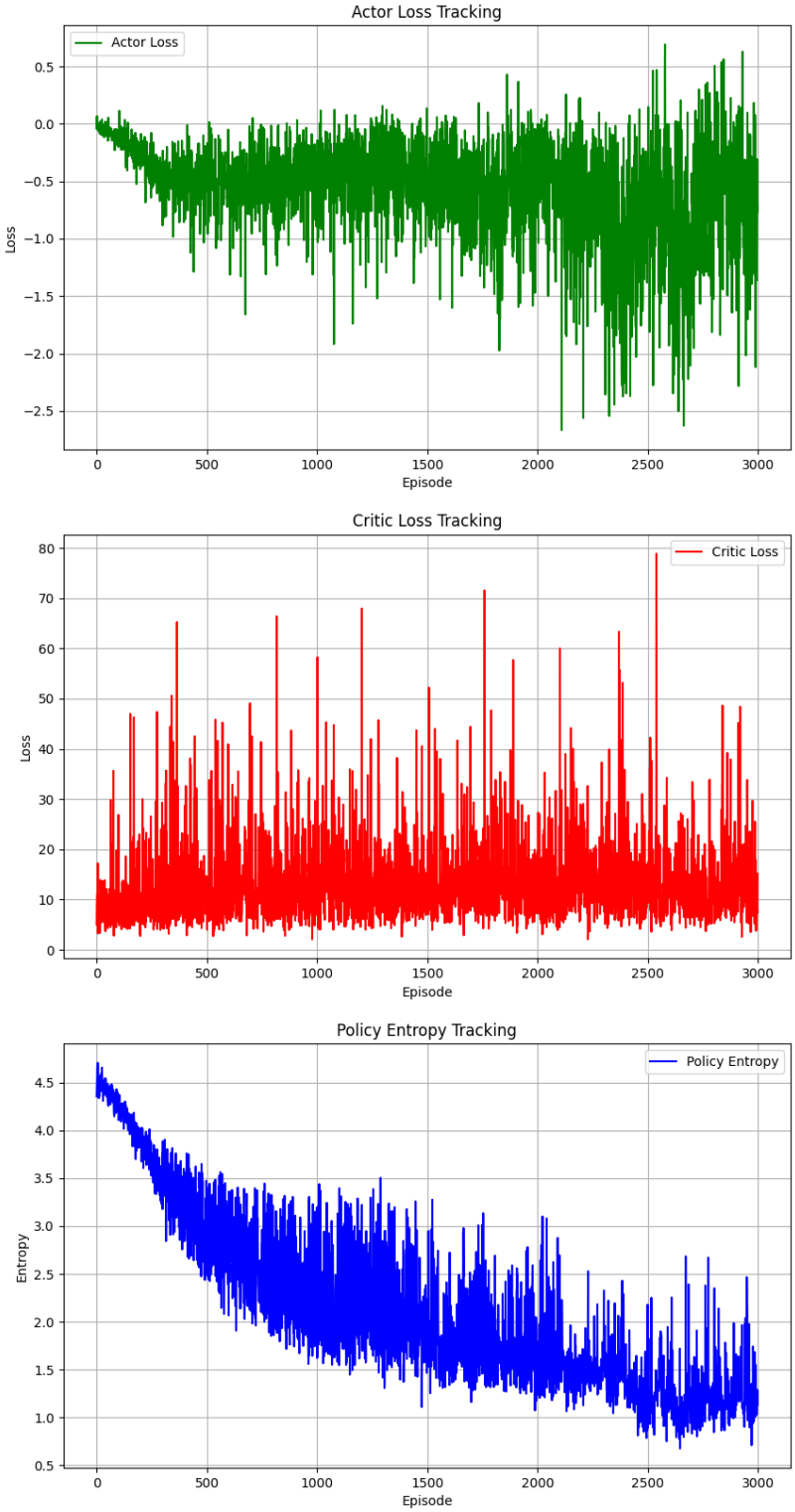


Figure 8: Loss tracker

3.4.2 Trend of Critic Loss

It mainly ranges from 5 to 15 with numerous spikes and a slight upward trend with training. This may be due to the highly non-stationary state transitions in Gomoku and the large variance of rewards, making it difficult for the Critic to accurately estimate the long-term value.

3.4.3 Trend of Policy Entropy

It starts at nearly 4.6, corresponding to a uniform random policy; it drops rapidly to about 2.0 in 0-1000 episodes with gradually increasing variance; it decreases slowly to about 1.1 in 1000-3000 episodes with gradually decreasing variance. Since entropy reflects uncertainty, it can be seen that the policy gradually changes from "strong exploration" to "stability" and still has a certain degree of randomness in the later stage without collapsing into a deterministic policy.

3.5 Solutions to Key Problems

First of all, by observing the loss curves of the Actor/Critic, we can find that they are very volatile, which is different from the previous supervised learning. After reflection, we find that the oscillation of Loss curves in RL is a normal phenomenon, which is essentially different from Supervised Learning.

- In supervised learning, the dataset D is fixed.
- In RL, the data is sampled by the current policy π_θ. As the network parameters θ are updated, the policy changes, and the distribution of the generated data also changes. The model is always chasing a moving target.

Reasons for the oscillation of Critic Loss
The target of the Critic is to minimize the TD Error:$L = (r + \gamma Q(s', a') - Q(s, a))^2$。

- When the Agent discovers a new high-value policy, the true return will increase sharply, leading to a huge error in the old Q-value estimation (Loss surges).
- Then the Critic slowly learns to fit this new policy, and the Loss decreases.
- Immediately after, the Agent discovers a better policy, and the process repeats. Therefore, the periodic spikes in Critic Loss are often a reflection of the model's "epiphany" rather than training failure.

Reasons for the oscillation of Actor Loss
Actor Loss is usually defined as $-\log \pi(a|s) \cdot A(s, a)$。

- Its essence is to push the policy towards the direction of high advantage ($A > 0$)
- As the policy becomes stronger, the originally high-advantage actions become "taken-for-granted" actions, and the value distribution of the advantage function A will change. This dynamic adjustment makes it difficult for Actor Loss to decrease monotonically like in supervised learning.

This naturally leads to the next question: what can the entropy curve reflect?

It can be seen from the expression of entropy that entropy reflects the randomness of the policy made by the model. In the initial stage of training, due to random initialization, the model may think that placing pieces anywhere is almost the same. In the later stage, the training tends to be stable, and the model's confidence in its own policy will increase, so the entropy will also decrease. Therefore, entropy is one of the indicators of system convergence.

At the same time, entropy is also easy to calculate. Through consulting materials, we can find that we can also evaluate the win rate index of the model:

- **Evaluation method**: Let the current model play against a fixed benchmark (e.g., random policy, greedy policy or early version of itself) at regular intervals (e.g., every 100 episodes).
- **Convergence signal**: The win rate curve shows an upward trend and finally stabilizes (e.g., stably above 90% or around the Nash equilibrium point of 50%). If the win rate still fluctuates greatly, it means that the model policy is still undergoing drastic adjustments.

However, this indicator has a disadvantage: it needs to play against other models after several rounds of training each time, which will lead to a long training time and lower time cost performance than entropy.

3.6 Model Optimization

In this experiment, model optimization is divided into two aspects: hyperparameter adjustment and architectural innovation.

3.6.1 Architectural Innovation

Main Architecture Description

The backbone network is used to extract high-level features from the original board state.

- **Input**: 12×12 board state tensor with dimension $(B, 1, 12, 12)$。
- **Stem Layer**: 3×3 convolution (64 channels) $\rightarrow$ BatchNorm $\rightarrow$ LeakyReLU。
- **Residual Tower**: Contains 5 standard Residual Blocks.
  - Block structure: $Conv3{\times}3 \rightarrow BN \rightarrow LeakyReLU \rightarrow Conv3{\times}3 \rightarrow BN \rightarrow Residualconnection \rightarrow LeakyReLU$。
- **Output**: Extracted feature map with dimension $(B, 64, 12, 12)$。

The **policy head** is responsible for outputting the probability distribution of piece placement in each step. The Attention mechanism is introduced to enhance the global receptive field.

- **Dimension Reduction Convolution**: 1×1 convolution, compressing the number of channels from 64 to 8 (Reduction Dim).
- **Attention Module**:

  - Flatten & Transpose → **RMSNorm** (Pre-Norm)。
  - **Multi-Head Attention (MHA)**: 4 Heads, Embedding Dim = 8。
  - Residual Add → **LayerNorm**。

- **Fully Connected Module (FC Block)**:
  - Flatten → Linear ($8 \times 12 \times 12 \rightarrow 512$) → LayerNorm → LeakyReLU → **Dropout (0.3)**。
  - Output Linear → dimension 144 (corresponding to 12×12 piece placement positions)。

The value head(Qhead) is responsible for evaluating the win rate $Q(s, a)$ of the current situation. Its structure is similar to the policy head but is activated by Tanh at the output end.

- **structure** Shares the design pattern with the Policy Head (Conv → Attention → FC).
- **output**: Scalar value activated by Tanh

optimizer selection: AdamW optimizer with weight decay = 1e-2.

Design Motivation Since our task is based on the Gomoku board, which is a 2D graph, the convolutional network can extract the position features in the 2D graph and the relationships between each piece. Therefore, we use a residual block as a combination of convolutional layers as the initial feature extractor, and use five such blocks in succession to fully extract features.

After compressing the features through a convolutional layer, we use the multi-head attention mechanism to extract the global features of the model. Note that since the previous convolutional layers have fully extracted the information of each position on the board and the correlations between each position, we do not need to insert positional encoding like the traditional transformer, but directly enter the multihead attention for forward propagation.

At the same time, we add residual connection blocks, which can ensure training stability, prevent problems such as gradient explosion or disappearance, and extract features better.

Since the number of training parameters is large, we need to design to prevent overfitting. This can be effectively prevented by changing the Adam optimizer to the AdamW optimizer. At the same time, we also introduce dropout in the final fully connected layer (in fact, the number of parameters of the final fully connected layer is large, which is $8 \times 12 \times 12 \times 512$)

To ensure training stability, we add RMSnorm before the multihead attention, BatchNorm after almost every convolutional layer, and layernorm after the fully connected layer.

Design Effect The final reduced param model achieves excellent performance in black-and-white games. It has a win rate of 93% against the original shallow network (500 rounds of games), as shown in the following figure (Figure 10: Game results with the original model):

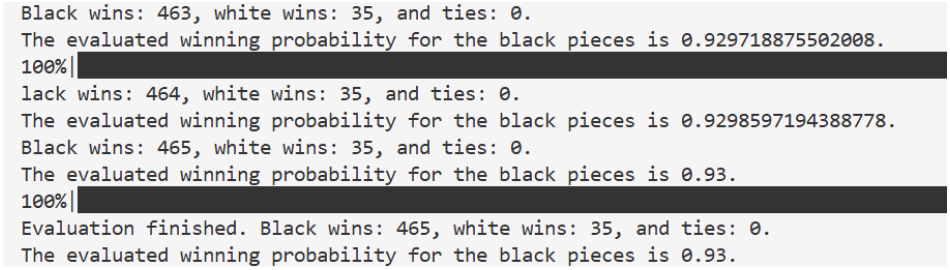


Figure 9: 与原始模型的对局结果

We also trained a version without the attention layer and increased the reduction dim of the dimension reduction convolution layer (i.e., the layer before the attention layer) to 32. The final game result is that the reduced param model won all 500 rounds, which reflects the very important role of the attention layer in this module

Figure 10: Caption

3.6.2 Hyperparameter Adjustment

After recognizing the superiority of this architecture, we also tried to increase the number of parameters of some layers in this architecture: change the reduction dim from 8 to 16, and change the p corresponding to dropout from 0.3 to 0.1. The final improved model has about 3M parameters and is trained for 5000 rounds. The final results are:

- The win rate of the model trained for 5000 rounds against the original model is 0.156
- The win rate of the model with this architecture trained for 4000 rounds against the original model is 0.448

This tells us that the increase in the number of model parameters does not necessarily mean the improvement of model capability. For this problem and this model architecture, the number of parameters of 1-2M is more appropriate. At the same time, too many training rounds cannot bring the improvement of the model architecture. In this problem, the model trained for 4000 rounds even performs better than the one trained for 5000 rounds, and too many training rounds may lead to overfitting.

This can also be explained from the training curve: the policy entropy remains at a relatively low level after about 2000 to 3000 rounds, and even rises a little bit at the end of 4000 to 5000 rounds, which indicates that too many training rounds lead to overfitting of the model in the scale of this problem.

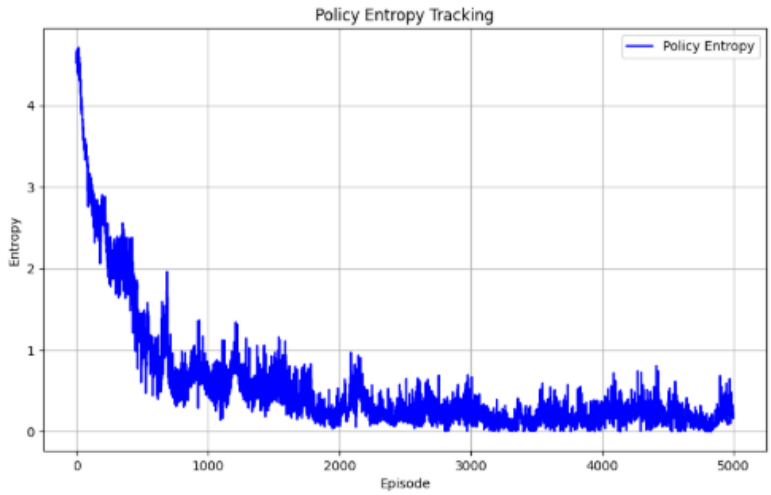


Figure 11: Caption